



THE CITY OF VAEL

FRAMING

01 – Repository and Architecture Review

How the City is framed today: shell, one live district, mock stores, and a half-wired service seam.

PREPARED FOR

Tony Grant, Owner
TSG Construction LLC

DOCUMENT TYPE

Milestone 1 deliverable
PRD acceptance pack

PREPARED BY

Appifest – Technical Audit
Continuing the existing prototype

DATE / METHOD

25 August 2026
Read-only inspection of the live tree

1. What Framing is

Framing is the application architecture: the React/TypeScript structure, how screens are nested, and which modules are allowed to own data. This inspection found a coherent city-and-district frame. It also found a second, newer frame (the service/provider seam) sitting in front of the older one, only partly connected to the walls.

2. Stack (as running)

LAYER	CHOICE	NOTES
UI	React 18 + TypeScript + Vite 5	<code>package.json</code> . Product name in npm is still <code>media-tech-exchange</code> – leftover from the Media & Technology origin.
Routing	react-router-dom v6	Flat route list in <code>src/App.tsx</code> – city shell, nested Media & Technology, legal, legacy redirects.
Styling	Tailwind CSS + VAEL design tokens	<code>design/VAEL_DESIGN_SYSTEM.md</code> , <code>src/index.css</code> .
Client state	<code>AuthProvider</code> / <code>useAuth</code> only	No Redux/Zustand. TanStack Query is installed and wrapped but is not the primary data pattern.
Tests	vitest, logic modules only	55 <code>*.test.ts</code> files. Zero <code>.tsx</code> component tests, by established convention.
Backend at runtime	None	Every live write goes through a <code>mockX.ts</code> module into <code>localStorage</code> .

LINEAGE, NOT A SECOND PRODUCT

This repository was originally scaffolded from an unrelated construction-trades platform (“The Contractor Exchange”). `PLAN.md` is historical. No live code, users, or branding are shared with that product. Only patterns were carried forward. The PRD’s “continue, don’t rebuild” applies to *this* tree.

3. Folder map

PATH	ROLE
<code>src/pages/city/*</code>	City-wide Rooms: home, feed, search, go visible, districts, account, notifications, messages, Extended VAEL demo

PATH	ROLE
<code>src/pages/mediaTechnology/*</code>	District layout wrapper only
<code>src/pages/*.tsx</code> (legacy flat)	The actual Media & Technology marketplace: Board, Veil, Handshake, profile, admin, community
<code>src/components/city/*</code>	City chrome (nav, district cards, homepage)
<code>src/lib/districts.ts</code>	The district registry – static data, no storage
<code>src/lib/mock*.ts</code>	One module per localStorage domain. The only legal readers/writers of those keys
<code>src/lib/{auth,vael,connection,...}/</code>	Thirteen service/provider seams prepared for a future Foundation cutover
<code>src/lib/matching.ts</code>	Percentage-fit engine – Media & Technology fields, not generic
<code>supabase/migrations/</code>	Foundation drawings (0001-0020). Not what the running app uses
<code>src/data/listings.ts</code>	Dead scaffolding. Talks to a table named <code>exchange_listings</code> that does not exist in any migration. Unused by screens

4. City shell vs district Rooms

The City is a parent shell. Districts are Rooms inside it. The registry in `src/lib/districts.ts` is the map of lots:

SLUG	NAME	STATUS	ROUTE
<code>media-technology</code>	Media & Technology	LIVE	<code>/media-technology</code>
<code>contractor</code>	Contractor	COMING SOON	<code>/districts/contractor</code>
<code>trucking</code>	Trucking	COMING SOON	<code>/districts/trucking</code>
<code>nursing-healthcare</code>	Nursing / Healthcare	COMING SOON	<code>/districts/nursing-healthcare</code>
<code>equipment</code>	Equipment	EARLY ACCESS	<code>/districts/equipment</code>
<code>government</code>	Government	EARLY ACCESS	<code>/districts/government</code>
<code>real-estate</code>	Real Estate	EARLY ACCESS	<code>/districts/real-estate</code>
<code>legal-finance</code>	Legal & Finance	EARLY ACCESS	<code>/districts/legal-finance</code>

Non-live lots all render `DistrictPlaceholder.tsx`, which states plainly: this district is not open; Media & Technology is the only fully working district. Interest is stored locally via `mockEarlyAccess.ts` – not a real waitlist backend.

PRD MISMATCH – FRAMING OF THE LOTS

The PRD names Construction Exchange, Residential, and Commercial. The registry names Contractor, and has no Residential or Commercial lots. Real Estate is early access, not a homeowner flow. A second live district should be a *new parallel folder/layout*, not a generic rewrite of Media & Technology, unless the Owner explicitly orders that refactor. That rule is already written in the project’s own architecture file.

5. Routing

City (CityLayout)

/ home · /feed · /search · /go-visible · /districts · /districts/:slug · /today · /concierge · /account · /account/visibility · /account/notifications · /notifications · /messages · /extended-vael (plans / checkout / success)

Media & Technology (MediaTechnologyLayout)

/media-technology plus auth, join, veil, post-opportunity, board, connections, match/buyer dashboards, profile, admin, how-it-works, community.

Legal (city chrome, not district)

/legal/terms · /legal/privacy · /legal/sms-terms

Legacy redirects

Every pre-city flat path (/board , /connections/:id , /profile/:username , ...) redirects into /media-technology/... so old bookmarks still work.

6. The mockX.ts ownership rule

Each `src/lib/mock*.ts` file is the only place that may read or write its localStorage key. Pages must go through that module. This rule is documented in each file header and in the project’s architecture notes. It is Framing discipline, and it is generally followed.

MODULE	KEYS OWNED
mockAuth.ts	mtx_mock_accounts_v1 , mtx_mock_session_v1 , mtx_mock_pending_reset_v1
mockListings.ts	mtx_mock_listings_v1

MODULE	KEYS OWNED
<code>mockConnections.ts</code>	<code>mtx_mock_connections_v1</code> , <code>mtx_mock_messages_v1</code> , <code>mtx_mock_reports_v1</code>
<code>mockProfiles.ts</code>	<code>mtx_mock_profile_details_v1</code>
<code>mockIdentity.ts</code>	<code>mtx_mock_identity_v1</code>
<code>mockCommunity.ts</code>	posts, comments, reactions, saves, seeded, show-samples keys
<code>mockDocuments.ts</code>	<code>mtx_mock_documents_v1</code>
<code>mockAttachments.ts</code>	<code>mtx_mock_attachments_v1</code>
<code>mockNotifications.ts</code> + preferences + SMS consent	three separate keys
<code>mockBlockedUsers.ts</code>	<code>mtx_mock_blocked_users_v1</code>
<code>mockAdmin.ts</code> / <code>mockCampaigns.ts</code>	badges, activity, campaigns
<code>mockEarlyAccess.ts</code> / <code>mockAccessIntent.ts</code>	city-prefixed keys (not <code>mtx_</code>)
<code>mockBadges.ts</code> / <code>mockSearch.ts</code>	None – computed / aggregated reads only

7. The second frame: service / provider seam

Stage 15 added a provider boundary in front of many domains so a future Foundation cutover could flip a flag instead of rewriting screens. Each domain has `XService.ts` plus local and (usually) Supabase providers. Authority is summarized in `src/lib/authority/registry.ts`.

DOMAIN	WIRED INTO UI TODAY?	WHAT SCREENS ACTUALLY CALL
auth	Yes – session only	<code>useAuth</code> → <code>authService</code> . Many identity/directory calls still go to <code>mockAuth</code> directly (see 03).
profileDirectory	Yes	Public username/identity reads
profile	Yes – writes only	<code>ProfilePage</code> writes through <code>profileService</code> ; display reads still hit <code>mockProfiles</code>
documents	Yes	<code>ProfileDocumentsTab</code>
community	Yes	City feed save + district community composer/cards
vael	No	Board / VaelFlow still use <code>mockListings</code>
connection	No	Connections / ConnectionDetail / handshake button still use <code>mockConnections</code>

DOMAIN	WIRED INTO UI TODAY?	WHAT SCREENS ACTUALLY CALL
notification / messaging / media	No	Screens still use <code>mockNotifications</code> , <code>mockConnections</code> messages, <code>mockAttachments</code>
block / report	No	Connection-level block/report on the old mock path
moderation	No	No local provider by design. Refuses every call without a live Foundation

FRAMING RISK

Two generations of the same walls. The new seam is real code, with tests, but most of the Rooms people actually walk through still nail into the old studs. Flipping a `VITE*_PROVIDER=supabase` flag today would not move Board, Handshake, or Messages onto the Foundation – those screens never call the seam. Auth is the exception, and that exception is dangerous (see 03).

8. Core Rooms as framed today

Veil (24-hour default)

`visibilityPlans.ts` defines Free Daily VAEL at 24 hours. `visibilityLifecycle.ts` sets `DEFAULT_DURATION_HOURS = 24`. New listings in `mockListings.ts` receive `expiresAt` at creation. The board filters with `isLifecycleVisible()`. Re-veil reconciles open connections via `mockConnections.reconcileConnectionsAfterReVael`. Extended plans are catalog + demo checkout only.

Matching Board

`src/lib/matching.ts` scores offering vs seeking listings. Weights (when applicable, sum 100): Discipline 22, Skills 10, Tools 9, Certifications 6, Location 10, Remote/Onsite 6, Timing 11, Experience 9, Engagement 6, Budget proxy 5, Completeness proxy 6. Bands: Strong ≥ 80 , Good ≥ 60 , Possible ≥ 40 . This engine is Media & Technology's, not a city-wide engine. It is a protected file – do not change weights without Owner confirmation.

Handshake Room

Unified model in `mockConnections.ts`: source is `board_match` or `handshake`; status pending / declined / connected / closed. Full profiles and chat live on the connection detail screen after mutual accept. `connectionService` exists and is unused by pages.

Profiles

`ProfileDetails` holds headline, bio, disciplines, skills, tools, location, rates, portfolio links, team. License, insurance, capability statement, and related credentials live on the documents tab (`DOC_TYPES` in `docTypes.ts`), not as first-class columns on the profile record. Completeness is a heuristic, not a verified-identity workflow.

Community

City homepage and `/feed` mix community cards with listings via `homeFeed.ts`. District community lives at `/media-technology/community`. Both go through `communityService` for posts/saves. Sample/demo posts are local-only by design.

9. Protected framing (do not casually restud)

Observed pattern across prior stages, not a mechanical lock: `matching.ts`, `Board.tsx` sort order, `LiveExchangeDemo.tsx`, and the Extended VAEL page set. Confirm with the Owner before changing them. `private/patent` is gitignored and must not be created or referenced.

10. Inspections (as designed in this repo)

The project's own verifier (`dev-agent/verify.mjs`) documents a vitest baseline of **586 tests / 57 files** and exactly **7 TypeScript errors** that are treated as a known fingerprint, not a license to add more:

- 3 in `src/data/listings.ts` – dead Supabase client possibly null / `never[]`
- 2 in `vite.config.ts` – Node `path` / `__dirname` types
- 2 in `vitest.config.ts` – same

This audit copy of the workspace had no `.git` directory and no successful `npm install` during the inspection window, so those commands were not re-run here. The fingerprint above is what the repository itself claims. A fresh clone with dependencies should re-confirm before Milestone 2 starts.

11. Framing verdict

KEEP THE FRAME

City shell + one live district + mock ownership + protected matching is a sound continue-don't-rebuild Framing. The unfinished work is (a) connecting the new service seam to the Rooms people already use, and (b) adding PRD districts as new parallel Rooms, not by melting Media & Technology into a generic blob.

THE CITY OF VAEL · Milestone 1 Technical Audit · 01 Framing. This document does not authorize development, connection of a live backend, or any Change Order.